

Project 1: This Ship is on *Fiiiiire!*

01:198:440

It is another day on the deep space vessel *Pteranodon*, and you are a lonely bot. You're responsible for the safety and security of the ship while the crew is in deep hibernation. As an example, in the case that there is a fire on the ship, you are responsible for pressing the button to trigger the fire suppression system. Of course, you have to get to it first.

1 The Ship

The layout of the ship (walls, hallways, etc) is on a square grid, generated in the following way:

- Start with a square grid, $D \times D$, of 'blocked' cells. Define the neighbors of cell as the adjacent cells in the up/down/left/right direction. Diagonal cells are not considered neighbors.
- Choose a square in the interior to 'open' at random.
- Iteratively do the following:
 - Identify all currently blocked cells that have exactly one open neighbor.
 - Of these currently blocked cells with exactly one open neighbor, pick one at random.
 - Open the selected cell.
 - Repeat until you can no longer do so.
- Identify all cells that are 'dead ends' - open cells with one open neighbor.
- Pick random dead ends and select a (random) closed neighbor of those dead ends to open, repeating this until the total number of dead ends has been cut by at least half.

Note 1: How big an environment D you can manage is going to depend on your hardware and implementation, but you should aim to generate data on as large an environment as is feasible.

Note 2: Your code for ship generation will be re-used on later projects.

2 The Bot

The bot occupies an open cell somewhere in the ship (to be determined shortly). The bot can move to one adjacent cell every time step (up/down/left/right).

3 The Fire

At a random open cell, a fire starts. Every time step, the fire has the ability to spread to adjacent open cells. The fire cannot spread to blocked cells. The fire spreads according to the following rules: At each timestep, a non-burning cell catches on fire with the probability $1 - (1 - q)^K$:

- q is a parameter between 0 and 1, defining the *flammability* of the ship.

- K is the number of *currently burning neighbors* of this cell.

Every timestep, each cell is updated according to the above rules.

Note: If q is very close to 0, $1 - (1 - q)^K$ will be close to 0, and the fire spreads with very low probability. If q is close to 1, then $1 - q$ is close to 0, and $1 - (1 - q)^K$ is close to 1 - meaning that the fire spreads with very high probability. Additionally, the more neighbors a cell has that are on fire, the higher K is, and the closer $1 - (1 - q)^K$ will be to 1 - the faster the fire will spread.

4 The Button

Located somewhere in the ship, in an empty cell (to be determined shortly), is a button that triggers the fire suppression system, instantly choking the fire of oxygen and putting it out. This must be pressed in order to stop the fire.

5 The Task

At time $t = 0$, place the bot, the button, and the initial fire cell at random open cells in the ship (three distinct open cells). At every time step, $t = 1, 2, 3, 4, \dots$, the following happens in sequence:

- The bot decides which open neighbor to move to.
- The bot moves to that neighbor.
- If the bot enters the button cell, the button is pressed and the fire is put out - the task is completed.
- Otherwise, the fire advances.
- If at any point the bot and the fire occupy the same cell, the task is failed.

Ideally, the bot navigates to the button, avoids the fire, and puts the fire out. The goal of this assignment is to build and evaluate different strategies for governing how the bot decides where to go.

6 The Strategies

At any time, the bot can choose from the following actions: move to a neighboring open cell, or stay in place. The thing that differentiates the strategies is how the bot decides what to do.

For this assignment, you will implement and compare the performance of the following strategies.

- **Bot 1** - This bot plans the shortest path to the button, avoiding the initial fire cell, and then executes that plan. The spread of the fire is ignored by the bot.
- **Bot 2** - At every time step, the bot re-plans the shortest path to the button, avoiding the current fire cells, and then executes the next step in that plan.
- **Bot 3** - At every time step, the bot re-plans the shortest path to the button, avoiding the current fire cells *and any cells adjacent to current fire cells, if possible*, then executes the next step in that plan. If there is no such path, it plans the shortest path based only on current fire cells, then executes the next step in that plan.

- **Bot 4** - A bot of your own design.

Note: All four of these strategies will likely be based on similar algorithms (such as shortest path finding). It may be worth thinking about how you can employ modularity and code re-use to simplify your life. Honestly, it's always worth thinking about that.

The only restriction I am putting on Bot 4 is that it cannot be a version of Bot 3 that just puts a wider buffer on the fire cells. Do something more interesting. You are welcome and encouraged to talk to the TAs or myself to discuss your ideas or get inspiration.

Each bot will be evaluated on a number of ships, at various values of q , to establish its effectiveness.

7 Data and Analysis

In your writeup, consider and address the following:

- 1) Explain the design and algorithm for your Bot 4, being as specific as possible as to what your bot is actually doing. How does your bot factor in the available information to make more informed decisions about what to do next? *You should also endeavor to make your code efficient, and be clear about the steps you took to achieve that.*
- 2) For each bot, repeatedly generate test environments and evaluate the performance of the bot, for many values of q between 0 and 1. Graph the probability (or average frequency, rather) of the bot successfully putting out the fire. Be sure to repeat the experiment enough times to get accurate results for each bot, and each tested value of q . *Note, for sufficiently high q , the fire spreads so fast the problem really reduces to whether or not the bot started closer to the button than the fire. At this point and beyond, you shouldn't really see a distinction between the performance of the bots. You should try to identify the range of 'interesting' q and get a lot of data in this range. Be clear about the experiments and exploration you ran.*
- 3) When each bot fails or get trapped by the fire, *why* does it fail? Was there a better decision they could have made that would have saved them? Why or why not? Support your conclusions. *I mean this question at least partially diagnostically - if you code up your Bot 4 and it isn't good - it's worth trying to analyze why.*
- 4) Speculate on how you might construct the ideal bot. What information would it use, what information would it compute, and how? Especially if we take into account the fact that computation takes time, realistically, during which the fire could spread - how should you balance amount of computation vs 'intelligence' of the decision being made? *When is it intelligent to just make a break for it and hope?*

Bonus: *In the above, we considered randomly generated ship layouts. If you were to design the ship layout deliberately, maybe the bots would be more effective. The only restriction is that all open cells must be reachable from each other. Write a program to design/discover the most safe ship layout you can. Explain your logic and algorithm design choices, as well as giving the final ship layout. Justify, if you can, why it is as effective as it is.*